

# Experiment ‘p\_4\_3\_b\_every\_second\_letter’ Results

June 3, 2026

**Experiment outcome:** SUCCESS

**Bad responses:** 0

**Responses containing assume :** 0

**Responses containing expect :** 0

**Resolution attempts:** 3

**Hard fails (resolution):** 2

**Soft fails (resolution):** 0

**Verification attempts:** 1

## Problem Specification

**Problem name:** p\_4\_3\_b\_every\_second\_letter

**Natural language statement:** Write a method that takes a string and returns every second letter of the string.

**Method signature:** p\_4\_3\_b\_every\_second\_letter(s: string) returns (result: string)

## Ensures

- $|result| * 2 == |s| \vee |result| * 2 + 1 == |s|$
- forall  $i :: 0 \leq i < |result| \implies result[i] == s[2 * i]$

## GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

### Program → GenAI

You are given the following task to perform in Dafny:

Write a method that takes a string and returns every second letter of the string.

The signature should be:

`p_4_3_b_every_second_letter(s: string) returns (result: string)`

The method should respect the following contract:

`ensures |result| * 2 == |s| || |result| * 2 + 1 == |s|, ensures forall i :: 0 <= i < |`  
`↪ result| ==> result[i] == s[2 * i]`

Produce and show only the Dafny body of this method, including the curly braces that  
`↪` surround it. Do not show the signature nor contract. You must not use 'assume' nor  
`↪` 'expect' anywhere in your code.

### GenAI → Program

**System fingerprint:** null

**ID:** chatcmpl-Dmel8o0fJualOHi208gat73Gvs5OB

**Model:** gpt-5.5-2026-04-23

**Created at:** 1780488794

**Finish reason:** stop

**Usage:** 346 tokens in, and 4627 tokens out

```
{
  result := "";
  var pos := 0;

  while pos + 1 < |s|
    invariant 0 <= pos <= |s|
    invariant |result| * 2 == pos
    invariant forall i :: 0 <= i < |result| ==> result[i] == s[2 * i]
    decreases |s| - pos
  {
    ghost var prev := result;
    ghost var oldLen := |result|;
    ghost var oldPos := pos;

    assert oldLen * 2 == oldPos;
    assert oldPos + 1 < |s|;
    assert oldPos + 2 <= |s|;
    assert |prev| == oldLen;

    result := prev + s[oldPos..oldPos + 1];
```

```

pos := oldPos + 2;

assert |s[oldPos..oldPos + 1]| == 1;
assert s[oldPos..oldPos + 1][0] == s[oldPos];
assert |result| == oldLen + 1;
assert |result| * 2 == pos;

assert forall i | 0 <= i < |result| ensures result[i] == s[2 * i]
{
  if i < oldLen {
    assert result[i] == prev[i];
    assert prev[i] == s[2 * i];
  } else {
    assert i == oldLen;
    assert result[i] == s[oldPos..oldPos + 1][0];
    assert result[i] == s[oldPos];
    assert oldPos == 2 * i;
  }
}
}

if pos == |s| {
  assert |result| * 2 == |s|;
} else {
  assert pos < |s|;
  assert pos + 1 <= |s|;
  assert |s| <= pos + 1;
  assert pos + 1 == |s|;
  assert |result| * 2 + 1 == |s|;
}
}

```

## Program → GenAI

When using dafny resolve, the below error is emitted and resolve fails:

problem\_data/p\_4\_3\_b\_every\_second\_letter/program.dfy(43,2): Error: invalid

↪ UnaryExpression

```

|
43 | }
| ^

```

problem\_data/p\_4\_3\_b\_every\_second\_letter/program.dfy(42,4): Error: expected either 'by'

↪ or a semicolon following the assert expression

```

|
42 | }
| ^

```

2 parse errors detected in program.dfy

Correct the error in code by altering only the method body. Produce and show only the  
→ Dafny body, including the curly braces that surround it. Do not show the signature  
→ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

## GenAI → Program

**System fingerprint:** null

**ID:** chatcmpl-DmemUxcpHin54TDygEl9tmbIstiNF

**Model:** gpt-5.5-2026-04-23

**Created at:** 1780488878

**Finish reason:** stop

**Usage:** 1014 tokens in, and 2051 tokens out

```
{
  result := "";
  var pos := 0;

  while pos + 1 < |s|
    invariant 0 <= pos <= |s|
    invariant |result| * 2 == pos
    invariant forall i :: 0 <= i < |result| ==> result[i] == s[2 * i]
    decreases |s| - pos
  {
    ghost var prev := result;
    ghost var oldLen := |result|;
    ghost var oldPos := pos;

    assert oldLen * 2 == oldPos;
    assert oldPos + 1 < |s|;
    assert oldPos + 2 <= |s|;
    assert |prev| == oldLen;

    result := prev + s[oldPos..oldPos + 1];
    pos := oldPos + 2;

    assert |s[oldPos..oldPos + 1]| == 1;
    assert s[oldPos..oldPos + 1][0] == s[oldPos];
    assert |result| == oldLen + 1;
    assert |result| * 2 == pos;

    forall i | 0 <= i < |result|
      ensures result[i] == s[2 * i]
    {
      if i < oldLen {
        assert result[i] == prev[i];
        assert prev[i] == s[2 * i];
      } else {
        assert i == oldLen;
        assert result[i] == s[oldPos..oldPos + 1][0];
        assert result[i] == s[oldPos];
        assert oldPos == 2 * i;
      }
    }
  }
}
```

```

    }
  }
}

if pos == |s| {
  assert |result| * 2 == |s|;
} else {
  assert pos < |s|;
  assert pos + 1 <= |s|;
  assert |s| <= pos + 1;
  assert pos + 1 == |s|;
  assert |result| * 2 + 1 == |s|;
}
}

```

## Program → GenAI

When using dafny resolve, the below error is emitted and resolve fails:

```

problem_data/p_4_3_b_every_second_letter/program.dfy(23,14): Error: ghost variables such
  ↳ as prev are allowed only in specification contexts. prev was inferred to be ghost
  ↳ based on its declaration or initialization.

```

```

|
23 | result := prev + s[oldPos..oldPos + 1];
| ~~~~

```

```

problem_data/p_4_3_b_every_second_letter/program.dfy(23,23): Error: ghost variables such
  ↳ as oldPos are allowed only in specification contexts. oldPos was inferred to be
  ↳ ghost based on its declaration or initialization.

```

```

|
23 | result := prev + s[oldPos..oldPos + 1];
| ~~~~~

```

```

problem_data/p_4_3_b_every_second_letter/program.dfy(23,31): Error: ghost variables such
  ↳ as oldPos are allowed only in specification contexts. oldPos was inferred to be
  ↳ ghost based on its declaration or initialization.

```

```

|
23 | result := prev + s[oldPos..oldPos + 1];
| ~~~~~

```

```

problem_data/p_4_3_b_every_second_letter/program.dfy(24,11): Error: ghost variables such
  ↳ as oldPos are allowed only in specification contexts. oldPos was inferred to be
  ↳ ghost based on its declaration or initialization.

```

```

|
24 | pos := oldPos + 2;
| ~~~~~

```

4 resolution/type errors detected in program.dfy

Correct the error in code by altering only the method body. Produce and show only the  
 ↳ Dafny body, including the curly braces that surround it. Do not show the signature

↪ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

## GenAI → Program

**System fingerprint:** null

**ID:** chatcmpl-Dmen8JD6SnUxl3xblpkvnskBIxN2A

**Model:** gpt-5.5-2026-04-23

**Created at:** 1780488918

**Finish reason:** stop

**Usage:** 2090 tokens in, and 1021 tokens out

```
{
  result := "";
  var pos := 0;

  while pos + 1 < |s|
    invariant 0 <= pos <= |s|
    invariant |result| * 2 == pos
    invariant forall i :: 0 <= i < |result| ==> result[i] == s[2 * i]
    decreases |s| - pos
  {
    var prev := result;
    var oldLen := |result|;
    var oldPos := pos;

    assert oldLen * 2 == oldPos;
    assert oldPos + 1 < |s|;
    assert oldPos + 2 <= |s|;
    assert |prev| == oldLen;

    result := prev + s[oldPos..oldPos + 1];
    pos := oldPos + 2;

    assert |s[oldPos..oldPos + 1]| == 1;
    assert s[oldPos..oldPos + 1][0] == s[oldPos];
    assert |result| == oldLen + 1;
    assert |result| * 2 == pos;

    forall i | 0 <= i < |result|
      ensures result[i] == s[2 * i]
    {
      if i < oldLen {
        assert i < |prev|;
        assert result[i] == prev[i];
        assert prev[i] == s[2 * i];
      } else {
        assert i == oldLen;
        assert result[i] == s[oldPos..oldPos + 1][0];
        assert result[i] == s[oldPos];
        assert oldPos == 2 * i;
      }
    }
  }
}
```

```

    }
}

if pos == |s| {
  assert |result| * 2 == |s|;
} else {
  assert pos < |s|;
  assert pos + 1 <= |s|;
  assert |s| <= pos + 1;
  assert pos + 1 == |s|;
  assert |result| * 2 + 1 == |s|;
}
}

```

## Final Program

```

method p_4_3_b_every_second_letter(s: string) returns (result: string)
  ensures |result| * 2 == |s| || |result| * 2 + 1 == |s|
  ensures forall i :: 0 <= i < |result| ==> result[i] == s[2 * i]
{
  result := "";
  var pos := 0;

  while pos + 1 < |s|
    invariant 0 <= pos <= |s|
    invariant |result| * 2 == pos
    invariant forall i :: 0 <= i < |result| ==> result[i] == s[2 * i]
    decreases |s| - pos
  {
    var prev := result;
    var oldLen := |result|;
    var oldPos := pos;

    assert oldLen * 2 == oldPos;
    assert oldPos + 1 < |s|;
    assert oldPos + 2 <= |s|;
    assert |prev| == oldLen;

    result := prev + s[oldPos..oldPos + 1];
    pos := oldPos + 2;

    assert |s[oldPos..oldPos + 1]| == 1;
    assert s[oldPos..oldPos + 1][0] == s[oldPos];
    assert |result| == oldLen + 1;
    assert |result| * 2 == pos;

    forall i | 0 <= i < |result|
      ensures result[i] == s[2 * i]
    {
      if i < oldLen {

```

```

        assert i < |prev|;
        assert result[i] == prev[i];
        assert prev[i] == s[2 * i];
    } else {
        assert i == oldLen;
        assert result[i] == s[oldPos..oldPos + 1][0];
        assert result[i] == s[oldPos];
        assert oldPos == 2 * i;
    }
}
}

if pos == |s| {
    assert |result| * 2 == |s|;
} else {
    assert pos < |s|;
    assert pos + 1 <= |s|;
    assert |s| <= pos + 1;
    assert pos + 1 == |s|;
    assert |result| * 2 + 1 == |s|;
}
}

```

## Total Token Usage

Input tokens: 3450

Output tokens: 7699

Reasoning tokens: 6128

Sum of 'total tokens': 11149

## Experiment Timings

**Overall Experiment** started at 1780488793763, ended at 1780488942356, lasting 148593ms (148.59 seconds)

**Iteration #1** started at 1780488793763, ended at 1780488877629, lasting 83866ms (83.87 seconds)

**Iteration #2** started at 1780488877630, ended at 1780488918309, lasting 40679ms (40.68 seconds)

**Iteration #3** started at 1780488918309, ended at 1780488942356, lasting 24047ms (24.05 seconds)